

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



HỆ ĐIỀU HÀNH

ĐỒ ÁN 1 – Xv6 và Unix Utilities

Giảng viên: **Thầy Lê Quốc Hòa**

Nhóm sinh viên thực hiện: **Nhóm 8**

22120008 Phạm Thiên An

22120071 Phan Bá Đức

22120215 Nguyễn Thị Mỹ

Thành phố Hồ Chí Minh, ngày 01 tháng 11 năm 2024

MỤC LỤC

MỤC LỤC	2
I. THÔNG TIN NHÓM.....	3
1. Thông tin thành viên :	3
2. Phân công:	3
II. NỘI DUNG.....	4
1. pingpong :	4
2. primes :	5
3. find :	9
4. xargs :	11
TÀI LIỆU THAM KHẢO.....	13

I. THÔNG TIN NHÓM

1. Thông tin thành viên :

STT	Họ và tên	MSSV
1	Phạm Thiên An	22120008
2	Phan Bá Đức	22120071
3	Nguyễn Thị Mỹ	22120215

2. Phân công:

STT	Họ và tên	MSSV	Phân công	Mức độ hoàn thành
1	Phạm Thiên An	22120008	pingpong	100%
			primes	100%
2	Phan Bá Đức	22120071	xargs	100%
			Tổng hợp code	100%
3	Nguyễn Thị Mỹ	22120215	find	100%
			Viết báo cáo	100%

II. NỘI DUNG

Xv6 là một hệ điều hành đơn giản, nhẹ nhàng, được phát triển tại MIT để phục vụ mục đích học tập và nghiên cứu về hệ điều hành.

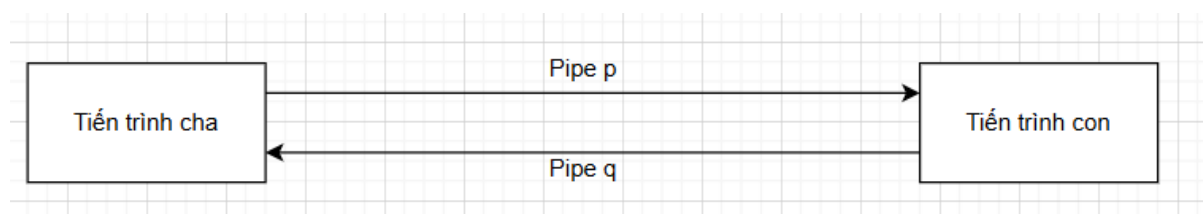
1. pingpong :

1.1. Mô tả yêu cầu :

Viết chương trình user-level sử dụng lệnh gọi hệ thống xv6 tới "ping-pong" một byte giữa hai tiến trình qua một cặp ống, mỗi ống một hướng. Tiến trình cha gửi một byte đến tiến trình con; tiến trình con in "<pid>: received ping", trong đó <pid> là ID tiến trình của nó, viết byte trên đường ống tới tiến trình cha và thoát; tiến trình cha đọc byte từ tiến trình con, in "<pid>: received pong" và thoát.

1.2. Phương pháp thực hiện :

1.2.1. Ý tưởng cho bài toán :



Tạo 2 đường ống để giao tiếp giữa 2 tiến trình.

1.2.2. Cách bước thực hiện :

- Bước 1 : Tạo 2 pipe :

- + 1 pipe theo hướng từ tiến trình cha sang tiến trình con.
- + 1 pipe theo hướng từ tiến trình con sang tiến trình cha.

- Bước 2 : Tạo tiến trình con bằng lệnh *fork*.

- Bước 3 : Kiểm tra các trạng thái của tiến trình.

- + Nếu tạo tiến trình thất bại, hiển thị thông báo và thoát.
- + Nếu đây là tiến trình con :
 - Đọc thông điệp từ pipe của cha sang con qua lệnh *read*.
 - In ra màn hình ID tiến trình con kèm tin nhắn "*received ping*".
 - Ghi lại thông điệp lên pipe của con sang cha qua lệnh *write*.
 - Kết thúc tiến trình sau khi đã gửi phản hồi.
- + Nếu là tiến trình cha :
 - Gửi thông điệp cho tiến trình con qua lệnh *write* để bắt đầu giao tiếp.

- Đọc thông điệp của tiến trình con qua lệnh *read*.
- In ra màn hình ID tiến trình cha kèm tin nhắn "*received pong*".

- **Bước 4** : Kết thúc chương trình khi hoàn tất giao tiếp giữa hai tiến trình.

1.3. Kết quả thực hiện :

- Kết quả thu được khi thực hiện gọi *pingpong.c* :

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ pingpong
4: received ping
3: received pong
$
```

- Kiểm tra bằng lệnh *make grade* :

```
== Test pingpong ==
$ make qemu-gdb
pingpong: OK (0.9s)
```

1.4. Khó khăn (nếu có) :

Giai đoạn đầu chưa hiểu về nguyên lý giao tiếp giữa 2 pipe, sau đó tự tìm hiểu và kiểm tra nên đã hiểu hơn về cách vận hành.

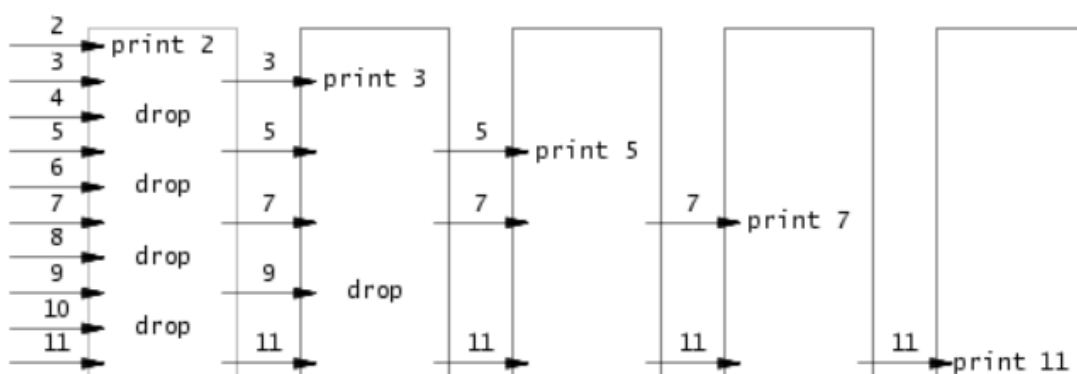
2. primes :

2.1. Mô tả yêu cầu :

Viết chương trình sàng nguyên tố đồng thời cho xv6 sử dụng *pipes*.

2.2. Phương pháp thực hiện :

2.2.1 Ý tưởng bài toán:



- Sàng nguyên tố là một thuật toán để tìm các số nguyên tố nhỏ hơn hoặc bằng một số n nhất định. Nó hoạt động bằng cách loại bỏ các bội số của mỗi số nguyên tố bắt đầu từ 2 và tiếp tục cho đến căn bậc hai của n .
- Trong chương trình này, mỗi số sẽ được truyền qua một *pipe* và loại bỏ nếu nó là bội số của một số nguyên tố đã tìm thấy.
- Cách hoạt động của chương trình:
 - + Dãy số tự nhiên được đẩy vào một *pipe*.
 - + Số đầu tiên trong luồng là số nguyên tố và được in ra.
 - + Tất cả các số là bội số của số này sẽ bị loại bỏ.
 - + Các số còn lại tiếp tục đi vào một *pipe* tiếp theo, lặp lại quá trình cho đến khi chỉ còn các số nguyên tố.

2.2.2. Cách bước thực hiện :

- Đầu tiên, xây dựng hàm sàng số nguyên tố *primes* với tham số đầu vào là một số :
 - + Chú ý khai báo thêm hàm *primes* sử dụng thuộc tính *noreturn* để trình biên dịch hiểu rằng hàm này sẽ không bao giờ quay lại hàm gọi nó, giúp tối ưu hóa việc gọi hàm và tránh lỗi đệ quy vô hạn.
 - + **Đọc số đầu tiên từ *pipe*:** Nếu không còn số nào để đọc, đóng *pipe* và thoát. Nếu đọc được số *num*, đây là số nguyên tố đầu tiên trong chuỗi.
 - + **In số nguyên tố:** Số đầu tiên đọc từ *pipe* là số nguyên tố vì nó không bị loại trừ trong bước sàng lọc trước đó.
 - + **Tạo *pipe* mới:** Mỗi số nguyên tố tìm được sẽ tạo một *pipe* mới để tiếp tục sàng lọc các số còn lại, loại bỏ các số chia hết cho số nguyên tố hiện tại.
 - + **Tạo tiến trình con:** Mỗi lần tìm được một số nguyên tố, tiến trình con sẽ xử lý sàng lọc số tiếp theo qua *pipe* mới.
 - + **Tiến trình con:** Đóng đầu ghi của *pipe* mới (chỉ đọc) và gọi đệ quy *primes* để tiếp tục sàng lọc các số không chia hết cho số nguyên tố hiện tại.
 - + **Tiến trình cha:** Đóng đầu đọc của *pipe* và bắt đầu đọc từng số từ số đầu tiên của *pipe*. Với mỗi số, nếu số đó không chia hết cho *num*, ghi số đó vào *pipe* mới *p[1]*.
 - + **Đóng và chờ tiến trình con:** Sau khi hoàn thành sàng lọc, tiến trình cha đóng đầu ghi *pipe*, chờ tiến trình con kết thúc và thoát.
- Sau đó, xây dựng hàm *main*:

+ **Tạo *pipe* đầu tiên.**

+ **Tạo tiến trình con đầu tiên:** Đóng đầu ghi của *pipe* (vì tiến trình con sẽ chỉ đọc) và gọi hàm *primes* truyền vào đầu đọc với dãy số từ 2 đến 280 để bắt đầu quá trình lọc các số nguyên tố từ *pipe*.

+ **Tiến trình cha:** Đóng đầu đọc của *pipe*, ghi các số từ 2 đến 280 vào đầu ghi của *pipe*, đóng đầu ghi khi đã ghi xong và chờ tiến trình con kết thúc trước khi thoát.

2.3. Kết quả thực hiện :

- Kết quả thu được khi thực hiện gọi *prime.c* :

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
prime 37
prime 41
prime 43
prime 47
prime 53
prime 59
prime 61
prime 67
prime 71
prime 73
prime 79
```

...

```
prime 131
prime 137
prime 139
prime 149
prime 151
prime 157
prime 163
prime 167
prime 173
prime 179
prime 181
prime 191
prime 193
prime 197
prime 199
prime 211
prime 223
prime 227
prime 229
prime 233
prime 239
prime 241
prime 251
prime 257
prime 263
prime 269
prime 271
prime 277
$
```

- Kiểm tra bằng lệnh *make grade* :

```
== Test primes ==
$ make qemu-gdb
primes: OK (1.7s)
```

2.4. Khó khăn (nếu có) :

Giai đoạn đầu chưa xử lý việc đóng, mở các *pipe* tốt dẫn đến rò rỉ bộ nhớ và nghẽn tài nguyên, chương trình không chạy tới 280 được, sau đó, đã cẩn thận việc đóng, mở hơn nên chương trình đã chạy tốt.

3. find :

3.1. Mô tả yêu cầu :

Viết 1 phiên bản đơn giản của chương trình tìm kiếm UNIX cho xv6. Nó sẽ có nhiệm vụ tìm kiếm tất cả các tệp trong cây thư mục có tên cụ thể.

3.2. Phương pháp thực hiện :

- Đầu tiên, ta sẽ viết 1 hàm con có chức năng tách tên file từ đường dẫn nhằm mục đích so sánh với tên file mà người dùng tìm kiếm.

- Sau đó, ta sẽ đi vào viết hàm *find*.

+ Kiểm tra xem đường dẫn có hợp lệ không : về chiều dài, về thông tin.

+ Mở đường dẫn, bắt đầu đọc và duyệt qua từng mục trong thư mục. Đồng thời nối tên mục vào biến *buf*.

+ Nếu mục đang xét là tập tin, ta sẽ thực hiện việc so sánh tên file của mục này (lấy từ hàm con đã viết ở trên) với tên file mà người dùng muốn tìm kiếm. Trong trường hợp đúng là tên mà người dùng tìm ta sẽ đánh dấu là đã tìm được ít nhất 1 file.

+ Còn nếu mục đang xét là thư mục (không xét thư mục hiện tại và thư mục cha) ta sẽ mở thư mục đó ra và tiếp tục gọi đệ quy để dò trong thư mục đó.

+ Sau khi hoàn thành, ta cần giải phóng tài nguyên để tránh gây cạn kiệt tài nguyên hệ thống.

3.3. Kết quả thực hiện :

- Trường hợp tìm file thông thường :

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
$
```

- Trường hợp file muốn tìm không tồn tại:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
$ find . c
find: file not found
$
```

- Kiểm tra bằng lệnh *make grade*:

```

135v04 linux-gnu objdump -C kernel/kernel | sed -i,51d0L 140L/0, 57 : / /
make[1]: Leaving directory '/home/nguyen_my/Project-Opertating-System'
== Test find, in current directory == find, in current directory: OK (1.7s)
== Test find, in sub-directory == find, in sub-directory: OK (0.9s)
== Test find, recursive == find, recursive: OK (1.1s)

```

3.4. Khó khăn (nếu có) :

4. xargs :

4.1. Mô tả yêu cầu :

Viết một phiên bản đơn giản của chương trình UNIX *xargs* cho xv6. Nó đọc các dòng từ đầu vào tiêu chuẩn rồi nối vào 1 câu lệnh để chạy (câu lệnh này là đối số của *xargs*).

4.2. Phương pháp thực hiện :

- **Bước 1** : Kiểm tra đối số của *xargs*, nếu đối số đầu (*argc*) vào bé hơn 2 (tức là chỉ gọi *xargs* mà không có thêm bất kỳ lệnh nào để thực thi) thì xuất ra *"Not have enough input"* (không có đủ input).

Ví dụ : *"xargs"* → (*argc* = 1) không hợp lệ.

"xargs echo bye" → (*argc* = 3) hợp lệ.

- **Bước 2** : Lưu lệnh thực thi (đối số của *xargs*) vào mảng *cmd_argv*.

Ví dụ : *"xargs echo bye"* → *cmd_argv* = *"echo bye"*.

- **Bước 3** : Đọc từng dòng của đầu vào tiêu chuẩn, với mỗi dòng ta thêm vào cuối mảng ban đầu *cmd_argv* (lưu lệnh thực thi). Sau đó gọi tiến trình con để thực thi lệnh được lưu trong *cmd_argv*.

Ví dụ : *echo hello | xargs echo bye* → *cmd_argv* = *"echo bye" + "hello" = "echo bye hello"*.

Nếu không gọi được tiến trình con thì xuất ra *"fork failed"*. Nếu là tiến trình cha thì đợi tiến trình con hoàn thành.

4.3. Kết quả thực hiện :

- Trường hợp thông thường :

```

$ echo hello | xargs echo bye
bye hello
$

```

- Kết hợp với hàm *find* và *grep* :

```
hart 1 starting
hart 2 starting
init: starting sh
$ echo Hello cac ban nho > b
$ mkdir a
$ echo Chung minh Hello ban nha > a/b
$ mkdir a/aa
$ echo Dung noi voi minh Hello > a/aa/b
$ find . b | xargs grep Hello
Hello cac ban nho
Chung minh Hello ban nha
Dung noi voi minh Hello
$
```

- Kiểm tra bằng *xargstest.sh* :

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sh < xargstest.sh
$ $ $ $ $ $ hello
hello
hello
$ $
```

- Kiểm tra bằng *make grade* :

```
== Test xargs ==
$ make qemu-gdb
xargs: OK (2.2s)
== Test xargs, multi-line echo ==
$ make qemu-gdb
xargs, multi-line echo: OK (1.1s)
```

4.4. Khó khăn (nếu có) :

TÀI LIỆU THAM KHẢO

- [1] hhp3. (2022, 07 22). *xv6 Kernel-35: File Descriptors and Open Files* | <https://www.youtube.com/watch?v=oWuVGDese4k>
- [2] Bell Labs and CSP Threads | <https://swtch.com/~rsc/thread/>
- [3] Lab: Xv6 and Unix utilities | <https://pdos.csail.mit.edu/6.1810/2024/labs/util.html>
- [4] Simple and straightforward tutorial on xargs! | <https://www.youtube.com/watch?v=-yJTjrv0mYU>